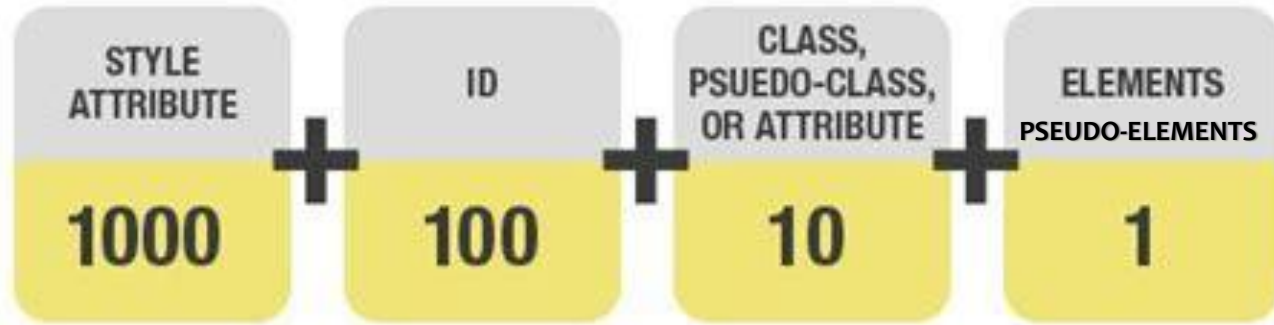


接下来主要内容

- 选择器
- 样式
- 层叠和继承
- 盒模型
- 布局

层叠和继承

- 层叠优先级算法
- **第三步**，根据选择器的特殊性 (Specificity) 进行优先级排序：



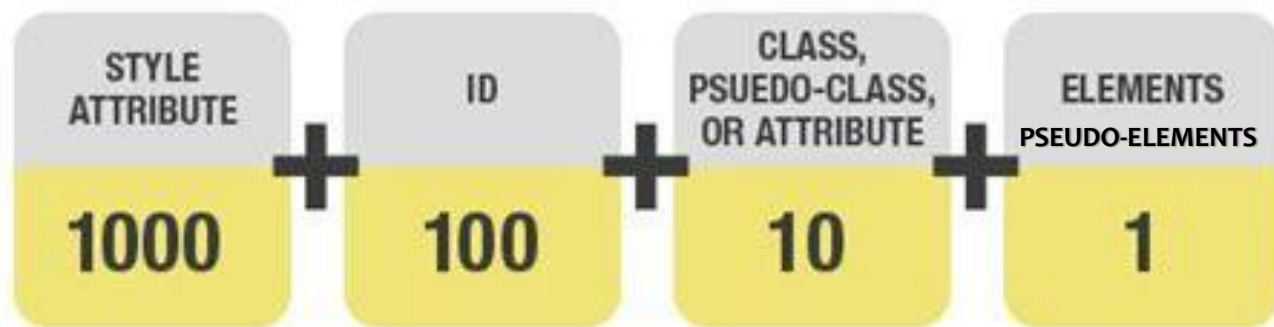
选择器特殊性是用四种不同的值来度量的，它们可以被认为是千位、百位、十位和个位：

- 千位：如果声明是在style 属性中该列加1分，否则为0（这样的声明没有选择器，特殊性总是1000）
- 百位：在选择器中每包含一个**ID选择器**就在该列中加1分
- 十位：在选择器中每包含一个**类、属性、伪类选择器**就在该列中加1分
- 个位：在选择器中每包含一个**元素选择**或**伪元素选择器**就在该列中加1分

注意：通用选择器(*), 组合选择器(+, >, ~, ' ') 和否定伪类(:not) 对特殊性无影响

层叠和继承

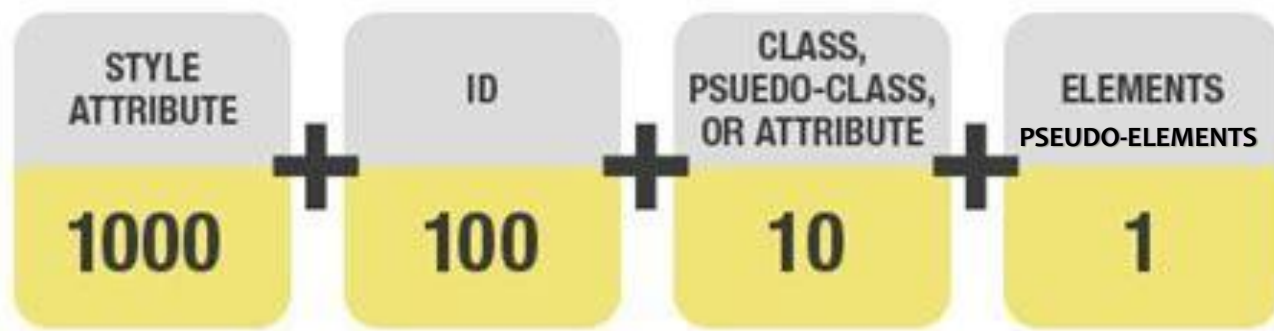
- 层叠优先级算法
- 第三步**，根据选择器的特殊性（Specificity）进行优先级排序：



选择器	元素选择器	千位	百位	十位	个位	合计值
h1		0	0	0	1	0001
#indentifier		0	1	0	0	0100
h1 + p::first-letter		0	0	0	3	0003
li > a[href*="zh-CN"] > .inline-warning		0	0	2	2	0022
This is a paragraph.		1	0	0	0	1000

层叠和继承

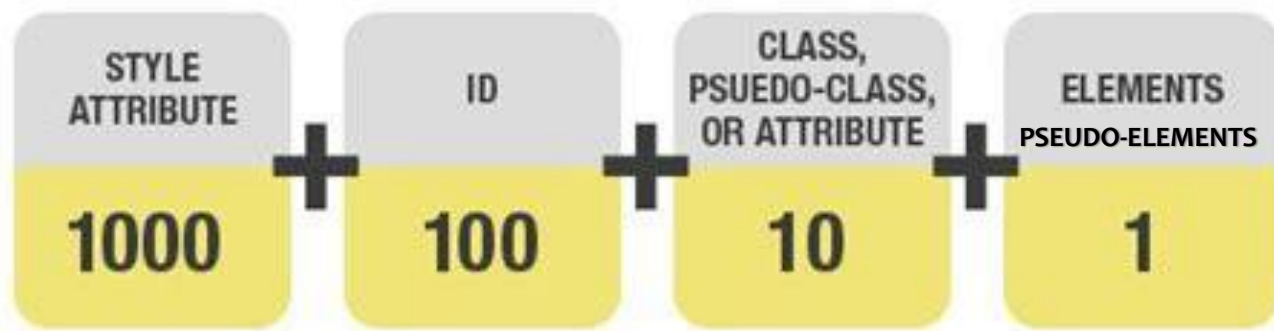
- 层叠优先级算法
- 第三步**，根据选择器的特殊性（Specificity）进行优先级排序：



选择器	千位	百位	十位	个位	合计值
h1	0	0	0	1	0001
#indentifier	0	1	0	0	0100
h1 + p::first-letter	0	0	0	3	0003
li > a[href*="zh-CN"] > .inline-warning	0	0	2	2	0022
This is a paragraph.	1	0	0	0	1000

层叠和继承

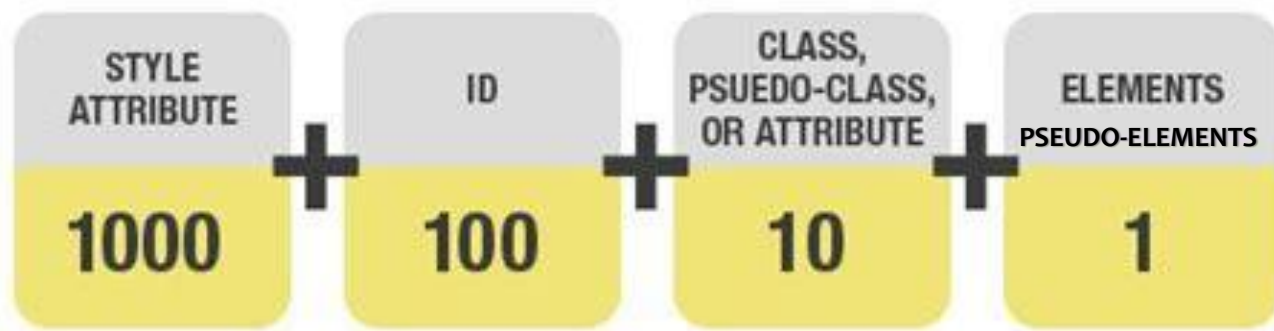
- 层叠优先级算法
- 第三步**，根据选择器的特殊性（Specificity）进行优先级排序：



选择器		百位	十位	个位	合计值
h1	h1元素选择器、p元素选择器、::first-letter伪元素选择器， +表示选择相邻兄弟	0	0	1	0001
#indentifier		1	0	0	0100
h1 + p::first-letter		0	0	3	0003
li > a[href*="zh-CN"] > .inline-warning		0	0	2	0022
This is a paragraph.		1	0	0	1000

层叠和继承

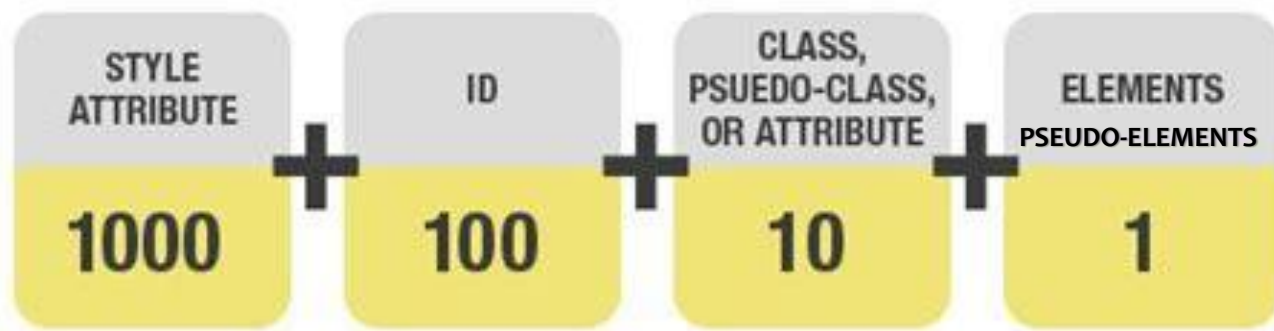
- 层叠优先级算法
- 第三步**，根据选择器的特殊性（Specificity）进行优先级排序：



选择器	千位	百位	十位	个位	合计值
li元素选择器、a元素选择器、 [href*="zh-CN"] 属性选择器、.inline-warning类选择器，>表示选择子元素					
h1	0	0	0	1	0001
#indent	0	1	0	0	0100
h1 + p::first-letter	0	0	0	3	0003
li > a[href*="zh-CN"] > .inline-warning	0	0	2	2	0022
This is a paragraph.	1	0	0	0	1000

层叠和继承

- 层叠优先级算法
- 第三步**，根据选择器的特殊性（Specificity）进行优先级排序：



选择器	千位	百位	十位	个位	合计值
h1	0	0	0	1	0001
#indentifier	0	1	0	0	0100
h1 + p::first-letter	0	0	0	3	0003
li > a[href*="zh-CN"] > .warning	0	0	2	2	0022
This is a paragraph.	1	0	0	0	1000

没有选择器, 规则在p元素的<style>属性里

层叠和继承

• 特殊性示例

选择器	特殊性的值
div ul li	0,0,0,3 3个元素选择器
div.aside ol li	0,0,1,3 1个类选择器, 3个元素选择器
a:hover	0,0,1,1 1个伪类选择器, 1个元素选择器
div.navlinks a:hover	0,0,2,2 1个类选择器, 1个伪类选择器, 2个元素选择器
.affix.top	0,0,2,0 多类选择器
input[type="text"]	0,0,1,1 1个属性选择器, 1个元素选择器
input[name="sex"][type="radio"]	0,0,2,1 2个属性选择器, 1个元素选择器
#title em	0,1,0,1 1个ID选择器, 1个元素选择器
h1#title em	0,1,0,2 1个ID选择器, 2个元素选择器
*	0,0,0,0 1个通用选择器

层叠和继承

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     h1#title em{color:green;}
7     #title em{color:red;}
8   </style>
9 </head>
10 <body>
11   <h1 id="title">Hello <em>Hello</em> Hello</h1>
12 </body>
13 </html>
```

显示效果

Hello *Hello* Hello

注意： 尽管h1#title和#title选择器表达的含义一样，且h1可以省略，但加上h1导致选择器的特殊性不同，最终生效的选择器也是不一样的

层叠和继承

- 层叠优先级算法
- **第三步**，根据选择器的特殊性（Specificity）进行优先级排序：

```
<a href="#">第一条应该是棕色</a>
  <!--适用第1行规则-->
<div class="demo">
  <input type="text" value="第二条应该是蓝色" />
  <!--适用第4、5行规则，第4行优先级高一-->
  <a href="#">第三条应该是黑色</a>
  <!--适用第1、2、3行规则，第3行优先级高一-->
</div>
<div id="demo">
  <a href="#">第四条应该是红色</a>
  <!--适用第1、2、6、7行规则，第7行优先级高一-->
</div>
```

如果有冲突，进入下一步→

第一条应该是棕色

第二条应该是蓝色

第三条应该是黑色

第四条应该是红色

```
a{color: brown;} /* 1. 特殊性值：0, 0, 0, 1*/
div a{color: green;} /* 2. 特殊性值：0, 0, 0, 2*/
.demo a{color: black;} /* 3. 特殊性值：0, 0, 1, 1*/
.demo input[type="text"]{color: blue;} /* 4. 特殊性值：0, 0, 2, 1*/
.demo *[type="text"]{color: grey;} /* 5. 特殊性值：0, 0, 2, 0*/
#demo a{color: orange;} /* 6. 特殊性值：0, 1, 0, 1*/
div#demo a{color: red;} /* 7. 特殊性值：0, 1, 0, 2*/
```

层叠和继承

- 层叠优先级算法
- **第四步**，根据CSS规则在样式文档中出现的位置（Source Order）进行优先级排序：
 - 如果多个相互竞争的选择器具有相同的重要性和特殊性，第四个因素将帮助决定哪一个规则获胜——**后面的规则将战胜先前的规则**

```
p {  
    color: blue;  
}  
  
/* This rule will win over the first one */  
p {  
    color: red;  
}
```

提示：

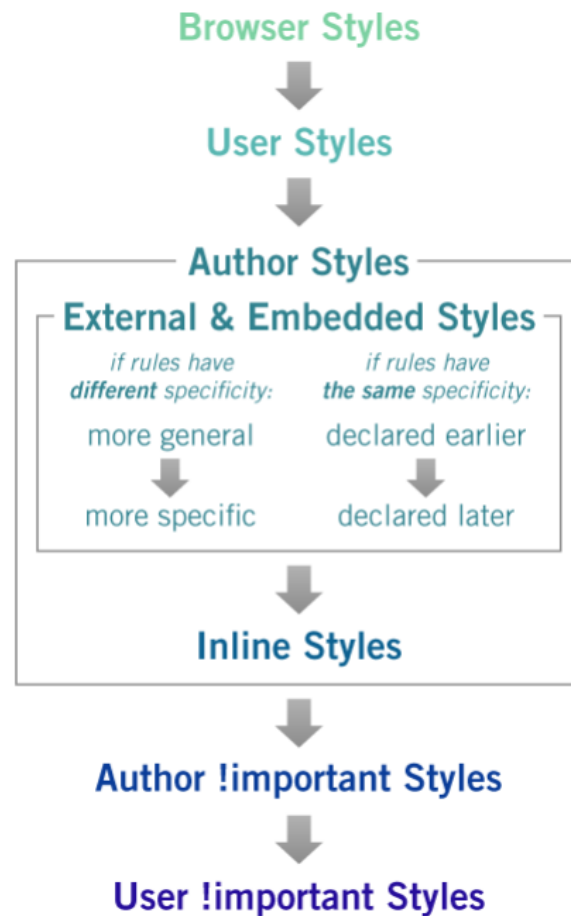
如果在样式文档中通过@import引入新的样式文档，则引入样式文档中的规则出现顺序要**先于**原始样式文档以及头部中声明的规则

```
@import url("style.css");
```

层叠和继承

• 层叠优先级算法

- **第一步**，针对某一元素的某一属性，列出所有给该属性指定值的CSS规则
- **第二步**，根据声明的重要性的来源进行优先级排序
- **第三步**，根据选择器的特殊性进行优先级排序
- **第四步**，根据CSS规则在样式文档中出现的位置进行优先级排序



层叠和继承

- 层叠优先级算法

```
<p class="better">This is a paragraph.</p>  
<p class="better" id="winning">One selector to rule them all!</p>
```

This is a paragraph.

One selector to rule them all!

color, padding属性不冲突, background-color、border属性冲突, 不冲突属性值直接应用, 冲突属性值通过优先级算法解决

```
#winning {  
  background-color: red;  
  border: 1px solid black;  
}  
  
.better {  
  background-color: gray;  
  border: none !important;  
}  
  
p {  
  background-color: blue;  
  color: white;  
  padding: 5px;  
}
```

重要: 在考虑层叠理论和什么样式优先于其他样式被应用时, 应该记住的是, **所有这些都发生在属性级别上——属性覆盖其他属性**, 但不会让整个规则凌驾于其他规则之上。当多个CSS规则匹配相同的元素时, 它们都被应用到该元素中。在此之后相互冲突的属性才会被评估, 以确定哪种样式会战胜其他样式

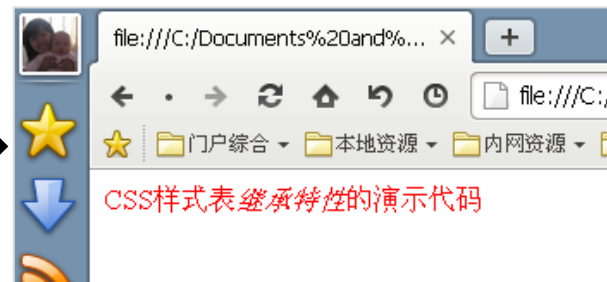
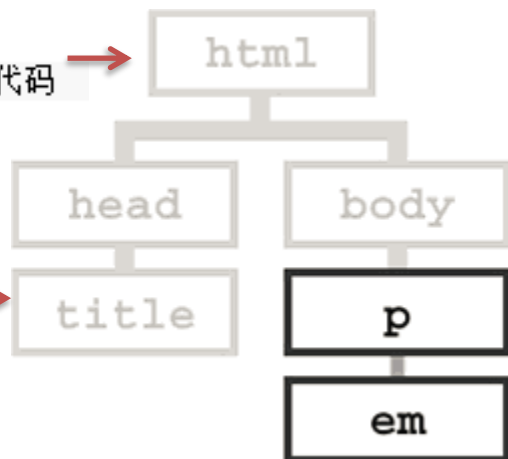
层叠和继承

• 继承 (Inheritance)

- 继承性是指被包在内部的标签默认拥有外部标签的样式性，即子元素可以继承父元素的属性
- 继承可以避免同样的内容重复声明，减少CSS文件大小，提升网页加载速度

```
1 <p>  
2 CSS样式表<em>继承特性</em>的演示代码  
3 </p>
```

```
1 <style>  
2 p { color:red; }  
3 </style>
```



我们并没有为em元素中的文字“继承特性”专门设定颜色，但是它默认继承了父元素p的颜色属性，默认为红色

层叠和继承

- 继承 (Inheritance)

- 在CSS规范中，每个属性定义中都指出了这个属性是默认继承的 (“Inherited: Yes”) 还是默认不继承的 (“Inherited: no”), 通常文本相关属性会默认继承，布局相关属性不默认继承，举例如下：

继承的属性	不继承的属性
color font (and related properties) letter-spacing line-height list-style (and related properties) text-align text-indent text-transform visibility white-space word-spacing	background (and related properties) border (and related properties) display float and clear height and width margin (and related properties) min- and max-height and -width outline overflow padding (and related properties) position (and related properties) text-decoration vertical-align z-index

层叠和继承

- 继承 (Inheritance)

– 在CSS规范中，每个属性定义中都指出了这个属性是默认继承的 (“Inherited: Yes”) 还是默认不继承的 (“Inherited: no”), 通常文本相关属性会默认继承，布局相关属性不默认继承

Appendix F. Full property table

规范中的定义参见W3C CSS 2.1 specification

<https://www.w3.org/TR/CSS21/propidx.html>

This appendix is informative, not normative.

Name	Values	Initial value	Applies to (Default: all)	Inherited?	Percentages (Default: N/A)	Media groups
'azimuth'	<angle> [[left-side far-left left center-left center center-right right far-right right-side] behind] leftwards rightwards inherit	center		yes		aural
'background-attachment'	scroll fixed inherit	scroll		no		visual
'background-color'	<color> transparent inherit	transparent		no		visual
'background-image'	<uri> none inherit	none		no		visual
'background-position'	[[[<percentage> <length> left center right] [<percentage> <length> top center bottom]?] [[left center right] [top center bottom]] inherit	0% 0%		no	refer to the size of the box itself	visual
'background-repeat'	repeat repeat-x repeat-y no-repeat inherit	repeat		no		visual
'background'	['background-color' 'background-image' 'background-repeat' 'background-attachment' 'background-position'] inherit	see individual properties		no	allowed on 'background-position'	visual
'border-collapse'	collapse separate inherit	separate	'table' and 'inline-table' elements	yes		visual
'border-color'	[<color> transparent]{1,4} inherit	see individual properties		no		visual
'border-spacing'	<length> <length>? inherit	0	'table' and 'inline-table' elements	yes		visual
'border-style'	<border-style>{1,4} inherit	see individual properties		no		visual
'border-top' 'border-right' 'border-bottom' 'border-left'	[<border-width> <border-style> 'border-top-color'] inherit	see individual properties		no		visual
'border-top-color' 'border-right-color' 'border-bottom-color' 'border-left-color'	<color> transparent inherit	the value of the 'color' property		no		visual
'border-top-style' 'border-right-style' 'border-bottom-style' 'border-left-style'	<border-style> inherit	none		no		visual

层叠和继承

- 继承 (Inheritance)

- 在CSS规范中，每个属性定义中都指出了这个属性是默认继承的 (“Inherited: Yes”) 还是默认不继承的 (“Inherited: no”), 通常文本相关属性会默认继承，布局相关属性不默认继承

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title></title>
  <style type="text/css">
    #p1 {color: red;}
    div {color: green;}
  </style>
</head>
<body>
  <p id="p1">
    This is an example.
    <div>
      <em>Yet another example!</em>
    </div>
  </p>
</body>
</html>
```

This is an example.

This is another example.

元素的color属性可以继承自其父节点<div>也可以继承自其祖先节点<p>

备注: 如果一个元素有两个祖先可以继承相同的属性，但属性值有所冲突，则在DOM树中，**离子孙最近的那个祖先的属性值会被继承下来**

层叠和继承

- CSS 中的 all 属性的不同取值对元素样式的影响。all 属性用于重置或继承元素的所有属性，常见取值包括 inherit、initial 和 unset：
 - 没有 all 属性：当没有应用 all 属性时，元素会按照所定义的样式规则渲染。
 - all: inherit;：所有样式属性将从父元素继承。如果父元素没有指定这些属性，浏览器将使用默认样式。这种情况下，元素的背景颜色和文本颜色将从父元素获取。
 - all: initial;：所有样式属性将重置为其初始值，即浏览器的默认样式。这个选项将忽略自定义样式，而回到浏览器的默认呈现。
 - all: unset;：此值将属性设置为 inherit 或 initial，取决于属性是否继承。非继承属性（如 background-color）将重置为初始值，继承属性（如 color）则会继承自父元素。
 - 注意：Internet Explorer 和 Safari 浏览器不支持 all 属性，因此在这些浏览器中可能会显示不同的效果。

层叠和继承

- CSS为控制继承提供了四种特殊的通用属性值：

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="utf-8">
5 <title>北邮网安院</title>
6 <style>
7 html {
8   font-size: small;
9   color: blue;
10 }
11
12 #ex1 {
13   background-color: yellow;
14   color: red;
15 }
16
17 #ex2 {
18   background-color: yellow;
19   color: red;
20   all: inherit;
21 }
22
23 #ex3 {
24   background-color: yellow;
25   color: red;
26   all: initial;
27 }
28
29 #ex4 {
30   background-color: yellow;
31   color: red;
32   all: unset;
33 }
34 </style>
35 </head>
36 <body>
37
38 <p>没有 all 属性:</p>
39 <div id="ex1">北邮网安院-web开发</div>
40
41 <p>all: inherit:</p>
42 <div id="ex2">北邮网安院-web开发</div>
43
44 <p>all: initial:</p>
45 <div id="ex3">北邮网安院-web开发</div>
46
47 <p>all: unset:</p>
48 <div id="ex4">北邮网安院-web开发</div>
49
50 <p><b>注意: </b> Internet Explorer 和 Safari 浏览器不支持 all 属性。</p>
51
52 </body>
53 </html>
54
```

没有 all 属性:

北邮网安院-web开发

all: inherit:

北邮网安院-web开发

all: initial:

北邮网安院-web开发

all: unset:

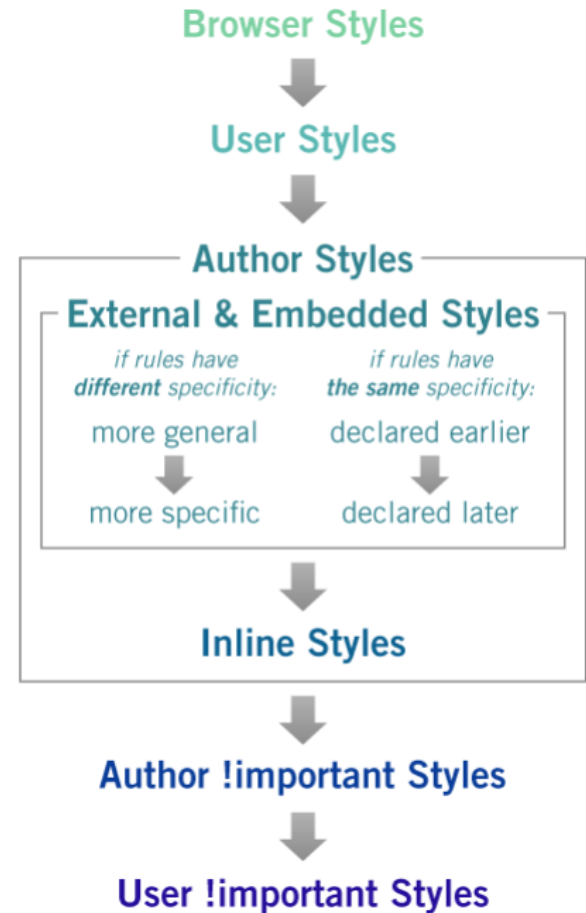
北邮网安院-web开发

注意: Internet Explorer 和 Safari 浏览器不支持 all 属性。

层叠和继承 - 小结

• 层叠优先级算法

- **第一步**，针对某一元素的某一属性，列出所有给该属性指定值的CSS规则
- **第二步**，根据声明的重要性的来源进行优先级排序
- **第三步**，根据选择器的特殊性进行优先级排序
- **第四步**，根据CSS规则在样式文档中出现的位置进行优先级排序



层叠和继承 - 小结

- 样式覆盖常见的5种冲突方式

① 引用方式冲突

- 行内样式 > (内部样式 = 外部样式)
- 如果内部样式与外部样式相同，**还是根据特殊性和源代码顺序进行比较**

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="external1.css">
<title>测试输入顺序</title>
<style>
p {color: black;}
#para1{color: red!important;}
.p1{color: blue!important;}
.p3{color: brown;}
#para4{color: rgb(114, 250, 121)!important;}
</style>
<link rel="stylesheet" href="external2.css">
</head>
<body>
<p class = "p1" id = "para1">paragraph 1</p>
<p class = "p2" id = "para2">paragraph 2</p>
<p class = "p3">paragraph 3</p>
<p class = "p4" id = "para4" style="color: pink!important;">paragraph 4</p>
</body>
</html>
```

欢迎使用

<> Untitled-1.html

external1.css X

```
1 p.p3 {color: aqua;}
```

欢迎使用

<> Untitled-1.html

external1.css

external2.css X

```
1 p.p3 {color: rgba(255, 0, 179, 0.76);}
```

paragraph 1

paragraph 2

paragraph 3

paragraph 4

Paragraph 3的颜色
是external2.css的选
择器设置的

er Security, BUPT #21

层叠和继承 - 小结

- 样式覆盖常见的5种冲突方式

② 继承方式冲突

- DOM树上最近的祖先获胜

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title></title>
  <style type="text/css">
    #p1 {color: red;}
    div {color: green;}
  </style>
</head>
<body>
  <p id="p1">
    This is an example.
    <div>
      <em>Yet another example!</em>
    </div>
  </p>
</body>
</html>
```

This is an example.

This is another example.

元素的color属性可以继承自其父节点<div>也可以继承自其祖先节点<p>

备注:如果一个元素有两个祖先可以继承相同的属性,但属性值有所冲突,则在DOM树中,离子孙最近的那个祖先的属性值会被继承下来

层叠和继承 - 小结

- 样式覆盖常见的5种冲突方式

- ③ 指定样式冲突

- 根据特殊性比较

选择器	权值
通配符	0
伪元素	1
元素选择器	1
class选择器	10
伪类	10
属性选择器	10
id选择器	100
行内样式	1000

层叠和继承 - 小结

- 样式覆盖常见的5种冲突方式

④ 继承样式和指定样式冲突

- 指定样式获胜

p1、p3-p4都存在继承样式和指定样式冲突问题

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="external1.css">
<title>测试输入顺序</title>
<style>
  div {color: teal;}
  #para1{color: red!important;}
  .p1{color: blue!important;}
  .p3{color: brown;}
  #para4{color: rgb(114, 250, 121)!important;}
  #para5{color: rgb(114, 250, 121)!important;}
</style>
<link rel="stylesheet" href="external2.css">
</head>
<body>
<div>
  <p class = "p1" id = "para1">paragraph 1</p>
  <p class = "p2" id = "para2">paragraph 2, 默认继承父元素颜色</p>
  <p class = "p3">paragraph 3</p>
  <p class = "p4" id = "para4" style="color: pink!important;">paragraph 4</p>
  <p class = "p5" id = "para5" style="color: pink!">paragraph 5</p>
</div>
</body>
</html>
```

paragraph 1

paragraph 2, 默认继承父元素颜色

paragraph 3

paragraph 4

paragraph 5



欢迎使用

Untitled-1.html

external1.css

external2.css ×

1 p.p3 {color: rgba(255, 0, 179, 0.76);}

层叠和继承 - 小结

- 样式覆盖常见的5种冲突方式

- ⑤ **!important**

- 如果一个样式用!important来声明，则这个样式会覆盖CSS中其他的任何非重要样式声明
 - eg: `strong{color:red !important;}`
 - 如果在同一个样式中出现了多个!important，则还是要根据特殊性和源代码顺序进行比较

层叠和继承 - 小结

- 样式覆盖常见的5种冲突方式

⑤ !important

- 如果在同一个样式中出现了多个!important, 则还是要根据特殊性和源代码顺序进行比较

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="external1.css">
<title>测试输入顺序</title>
<style>
p {color: black;}
#para1{color: red!important;}
.p1{color: blue!important;}
.p3{color: brown;}
#para4{color: rgb(114, 250, 121)!important;}
#para5{color: rgb(114, 250, 121)!important;}
</style>
<link rel="stylesheet" href="external2.css">
</head>
<body>
<p class = "p1" id = "para1">paragraph 1</p>
<p class = "p2" id = "para2">paragraph 2</p>
<p class = "p3">paragraph 3</p>
<p class = "p4" id = "para4" style="color: pink!important;">paragraph 4</p>
<p class = "p5" id = "para5" style="color: pink!">paragraph 5</p>
</body>
</html>
```

paragraph 1

paragraph 2

paragraph 3

paragraph 4

paragraph 5

行内样式 vs !important样式:

- 1、同样是important的规则, paragraph 4的颜色是内联样式胜出
- 2、Paragraph5的颜色则是#para5选择器设定的样式胜出, 因为其是重要规则

接下来主要内容

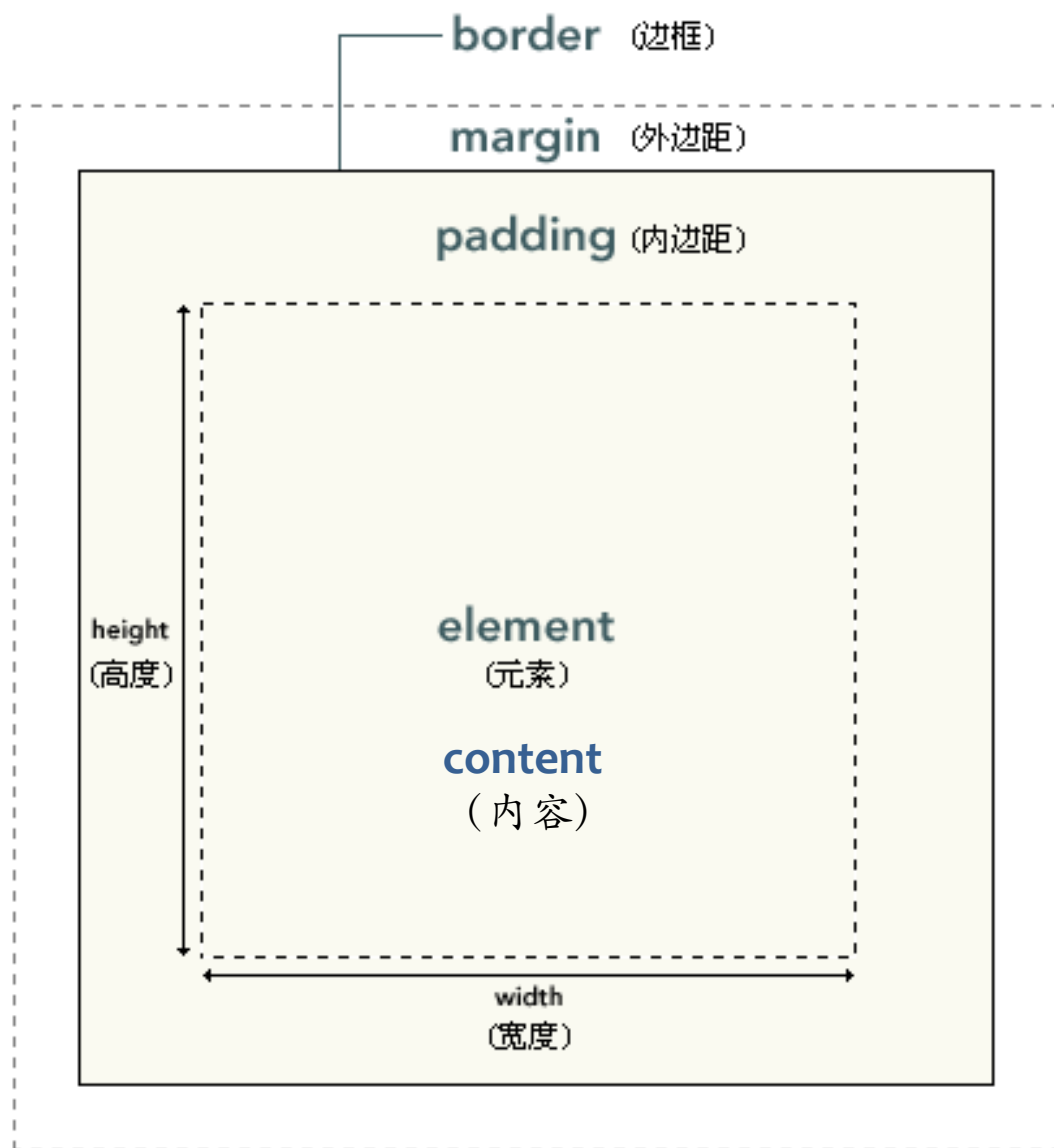
- 选择器
- 样式
- 层叠和继承
- 盒模型
- 布局

盒模型

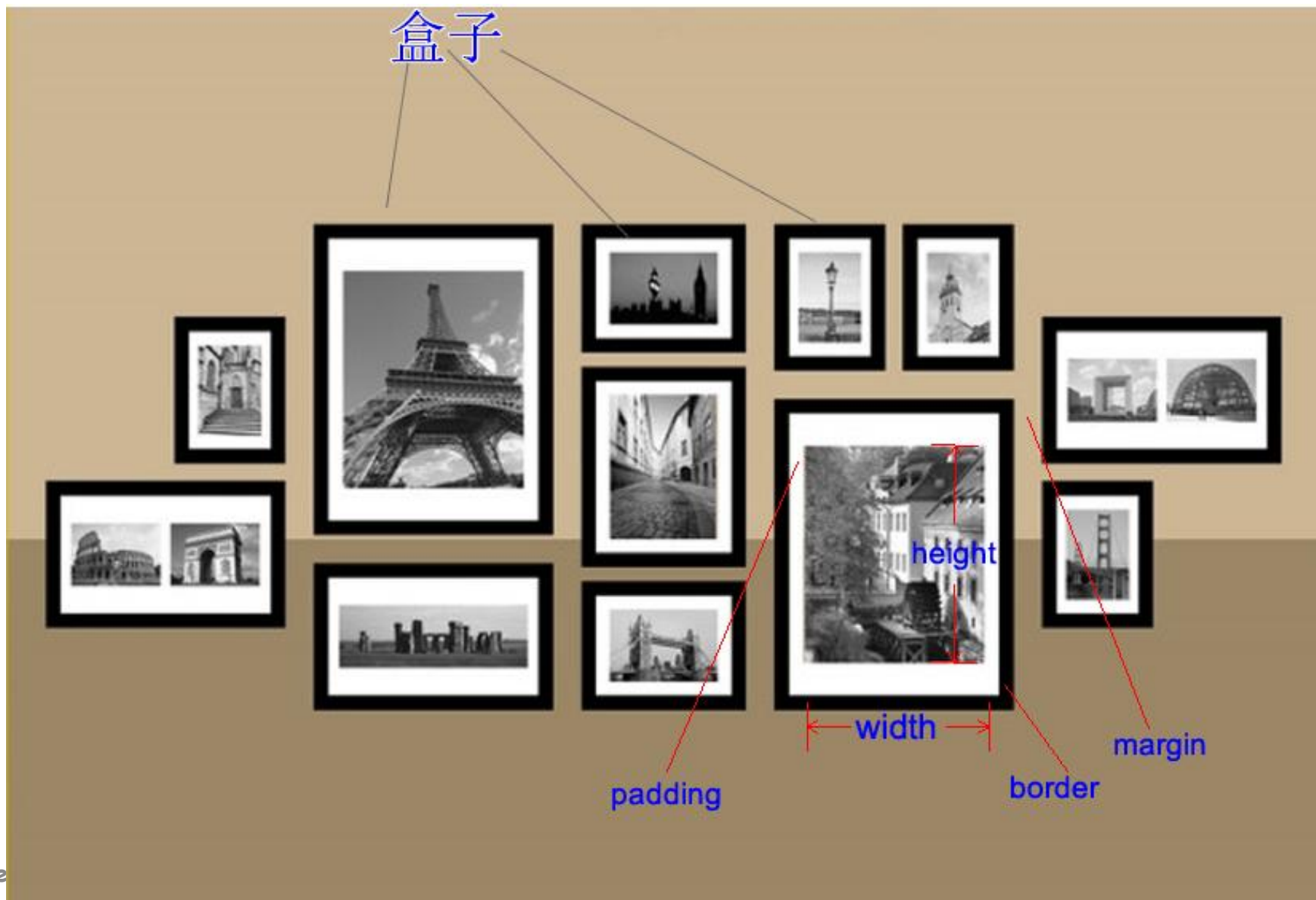
- **CSS盒模型(Box Model)**是网页布局的基础，每个元素被表示为一个矩形的方框，通过设置内容区域、内边距、边框和外边距，可用于浏览器渲染网页布局时计算盒子大小、相对位置等
- 在“**CSS盒模型**”理论中，所有页面中的**元素都可以看成一个盒子**，并且占据着一定的页面空间
- 一个页面由很多这样的盒子组成，这些盒子之间会互相影响，因此掌握盒子模型需要从两个方面来理解：**一是理解单独一个盒子的内部结构，二是理解多个盒子之间的相互关系**

盒模型

- 术语表
 - 盒模型box model
 - 元素element
 - 内容content
 - 内边距padding
 - 外边距margin
 - 边框border



盒模型



盒模型

表1 盒子模型4个属性

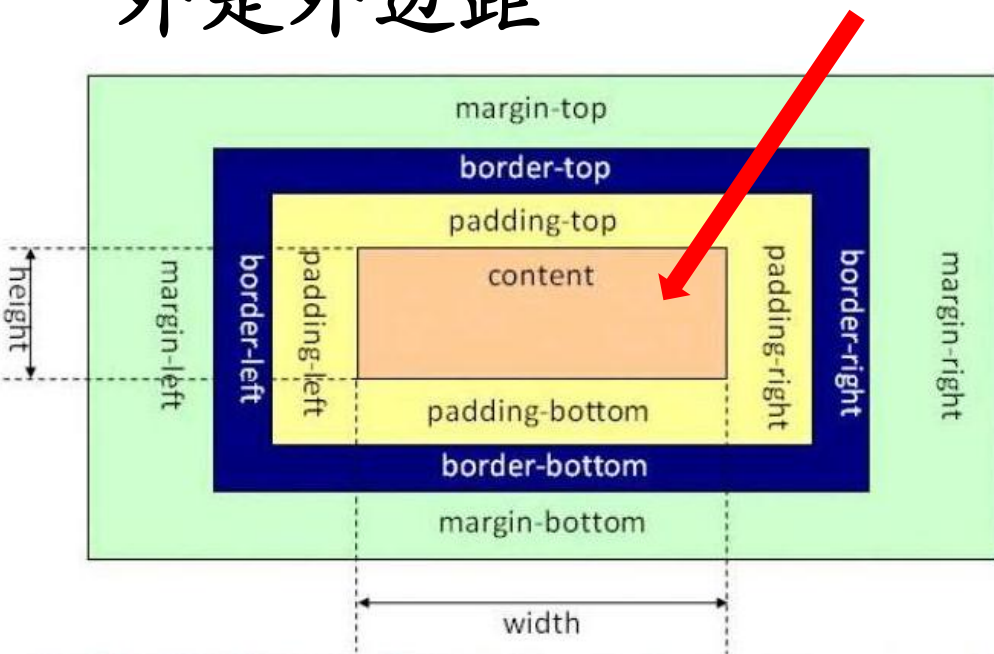
属性	说明
border	(边框) 元素边框
margin	(外边距) 用于定义页面中元素与元素之间的距离
padding	(内边距) 用于定义内容与边框之间的距离
content	(内容) 可以是文字或图片

盒模型

- 元素框的最内部分是实际的内容，直接包围内容的是内边距，内边距的边缘是边框，边框以外是外边距

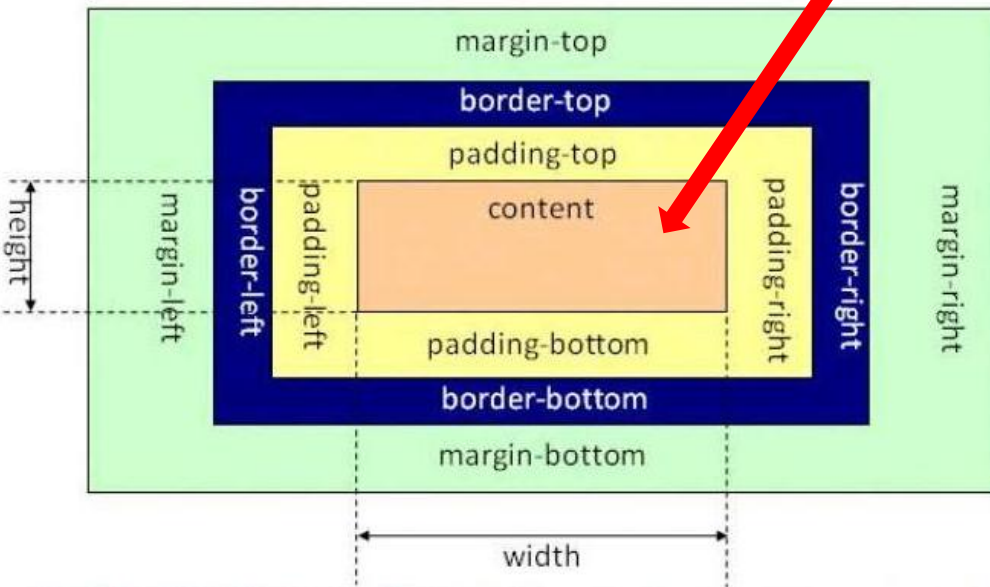
内容区

- 内容区是CSS盒子模型的中心，它呈现了盒子的主要信息内容，这些内容可以是文本、图片等多种类型
- 内容区是盒子模型必备的组成部分，其他的3部分都是可选的



盒模型

- 元素框的最内部分是实际的内容，直接包围内容的是内边距，内边距的边缘是边框，边框以外是外边距



内容区

- 内容区有3个属性：width、height和overflow
- 使用width和height属性可以指定盒子内容区的高度和宽度
- 当内容信息太多时，超出内容区所占范围时，可以使用overflow溢出属性来指定处理方法

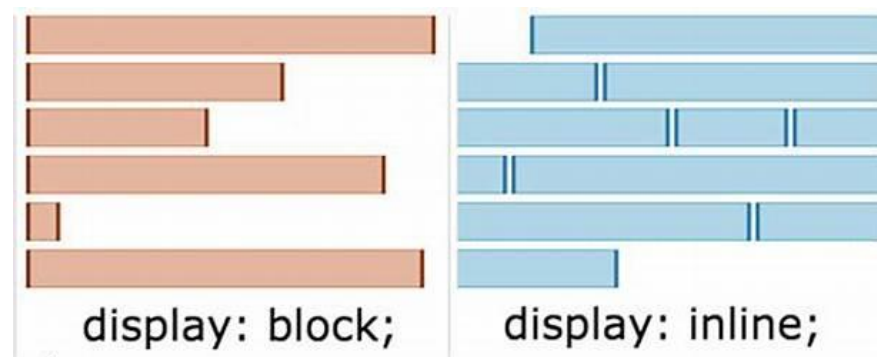
盒模型

- 内容区

- 注意：width和height是针对内容区的，并不包括padding部分
- 只有块级元素能设置width和height，内联元素无法设置width和height

块级元素VS内联元素

- 所有的HTML元素可以列为如下**三种**之一
 - **block块级元素**: 大块内容, 有height和width, 可以定义四个方向的margin
 - 例子<p>, <h1>, <blockquote>, , , <table>
 - **inline内联元素**: 少量内容, 没有height和width, 只可以定义左右两个方向的margin
 - 例子<a>, ,
 - 特殊: **inline block内联块元素**: 内联元素, 有height和width: <input>
 - **metadata元数据元素**: information about the page, usually not visible
 - 例子<title>, <meta>



块级元素VS内联元素

- 块级元素，默认从上到下排列，占据整行，可以设置宽和高
- 内联元素，默认从左向右排列，占用需要的位置，设置宽和高不生效，无法设定位置

设置宽和高的效果对比

在线代码测试

黑暗 黎明

显示效果

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     h1 {border:3px solid red;
7         height:5em;
8         width:75%;}
9     span {border:3px solid green;
10          height:5em;
11          width:75%;}
12   </style>
13 </head>
14 <body>
15   <h1>这是一级标题H1块级元素包裹的文字</h1>
16   <span>这是span内联元素包裹的文字</span>
17
18 </body>
19 </html>
```

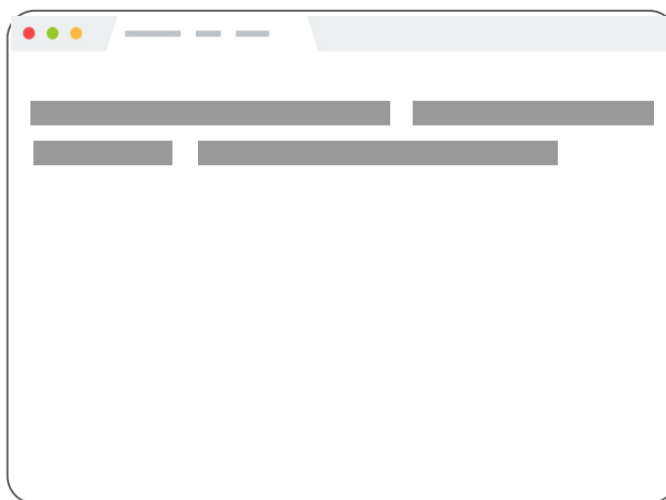
这是一级标题H1块级
元素包裹的文字

这是span内联元素包裹的文字

块级元素VS内联元素

- 块级元素，默认从上到下排列，占据整行，可以设置宽和高
- 内联元素，默认从左向右排列，占用需要的位置，设置宽和高不生效，无法设定位置

内联元素的默认排列效果



块级元素VS内联元素

- 块级元素，默认从上到下排列，占据整行，可以设置宽和高
- 内联元素，默认从左向右排列，占用需要的位置，设置宽和高不生效，无法设定位置

排列效果对比

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     h1,h2 {border:3px solid red;
7           height:2em;
8           width:100%;}
9     span {border:3px solid green;
10          height:5em;
11          width:100%;}
12   </style>
13 </head>
14 <body>
15   <h1>这是标题H1包裹的文字</h1>
16   <h2>这是标题H2包裹的文字</h2>
17   <span>这是span1包裹的文字</span>
18   <span>这是span2包裹的文字</span>
19   <span>这是span3包裹的文字</span>
20
21 </body>
22 </html>
```

显示效果

这是标题H1包裹的文字

这是标题H2包裹的文字

这是span1包裹的文字 这是span2包裹的文字 这是span3包裹的文字

块级元素VS内联元素

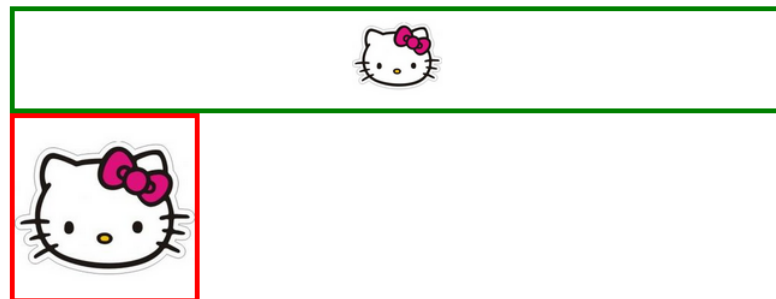
- 内联元素，要想设定位置，需要通过包裹它的元素设置

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     div {border:3px solid green;}
7     img.small {width: 50px;}
8     img.large {width: 100px;
9               border:3px solid red; }
10    div, img{text-align: center;}
11  </style>
12 </head>
13 <body>
14   <div >
15     
16   </div>
17   
18
19 </body>
20 </html>
```

显示效果



为设置text-align属性不生效
为<div>设置text-align属性生效

块级元素VS内联元素

- 内联元素，要想设定位置，需要通过包裹它的元素设置，**或者将其display属性设定为block**，占据整行的宽度，并可以设置宽度和高度。

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     a.inline {
7       border:3px solid red;
8       width:80%;
9       text-align: center;}
10    a.block {
11      display:block;
12      border:3px solid red;
13      width:80%;
14      text-align: center;}
15  </style>
16 </head>
17 <body>
18   <a class="inline" href="#" alt="">This is an inline-link</a>
19   <a class="block" href="#" alt="">This is a block-link</a>
20 </body>
21 </html>
```

显示效果

[This is an inline-link](#)

[This is a block-link](#)

为<a>设置text-align属性不生效
为具有display: block的<a>设置text-align属性生效

块级元素VS内联元素

- inline-block，内联块元素

- 或者CSS display属性设置成inline-block的任意元素

在线代码测试

黑暗 黎明

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml">
3 <head>
4   <title></title>
5   <style type="text/css">
6     img.small { width: 50px; }
7     img.large { width: 100px; }
8   </style>
9 </head>
10 <body>
11   
12   
13   
14   
15   
16
17 </body>
18 </html>
```

显示效果



可以设定宽度
占据需要的位置
从左向右排列

盒模型

- 溢出 (overflow)

- 利用overflow属性定义溢出元素内容区的内容会如何处理

/* 默认值，内容不会被修剪，呈现在元素框之外 */

overflow: visible;

/* 内容会被修剪，且其余内容不可见 */

overflow: hidden;

/* 内容会被修剪，浏览器会显示滚动条，以便查看其余内容 */

overflow: scroll;

/* 由浏览器定夺，如果内容被修剪，就会显示滚动条 */

overflow: auto;

visible (default)
Sed ut perspiciatis
unde omnis iste natus
error sit voluptatem
accusantium
doloremque
laudantium.

overflow: hidden
Sed ut perspiciatis
unde omnis iste natus
error sit voluptatem...

overflow: scroll
Sed ut perspiciatis
unde omnis iste
Sed ut perspiciatis
unde omnis iste
Sed ut perspiciatis

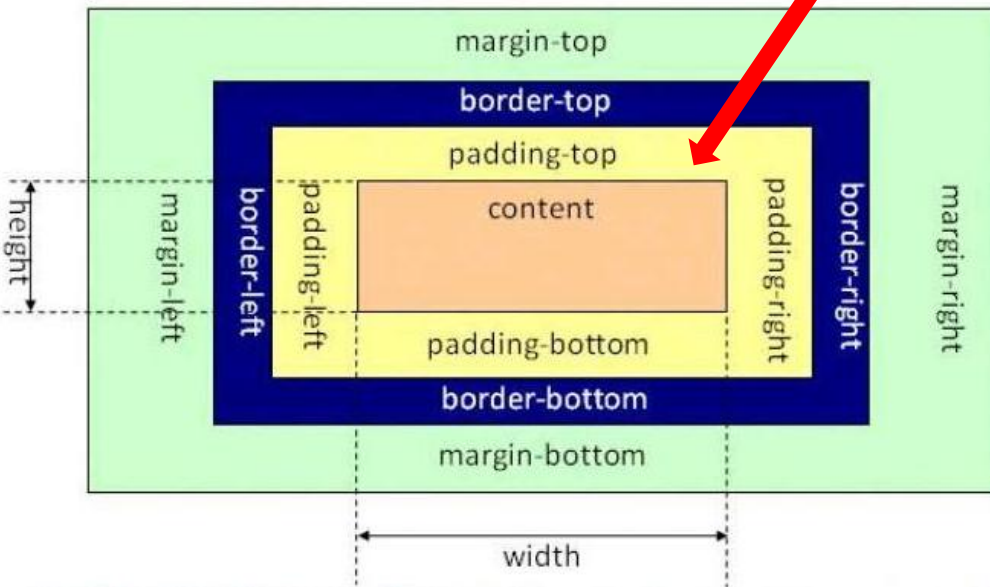
overflow: auto
Sed ut perspiciatis
unde omnis iste
Sed ut perspiciatis
unde omnis iste
Sed ut perspiciatis

盒模型

- 元素框的最内部分是实际的内容，直接包围内容的是内边距，内边距的边缘是边框，边框以外是外边距

padding 表示内边距

- 内边距指的是内容区和边框之间的空间，可以被看做是内容区的背景区域，也常被称为“补白”



盒模型

- 元素框的最内部分是实际的内容，直接包围内容的是内边距，内边距的边缘是边框，边框以外是外边距

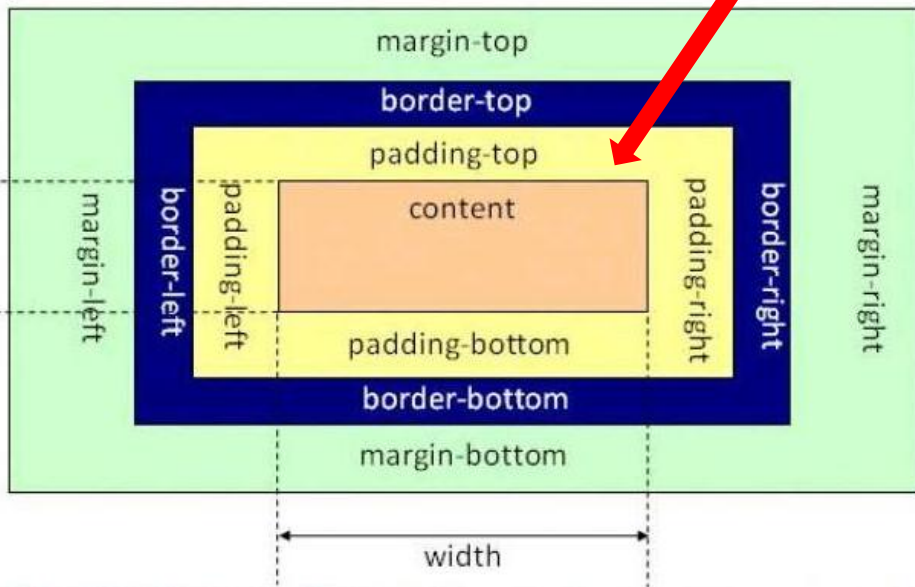
padding 表示内边距

- 大小可以通过简写属性padding一次设置所有四个边
- 也通过padding-top、padding-right、padding-bottom 和 padding-left 属性一次设置一边

```
h1 {padding: 10px;}
```

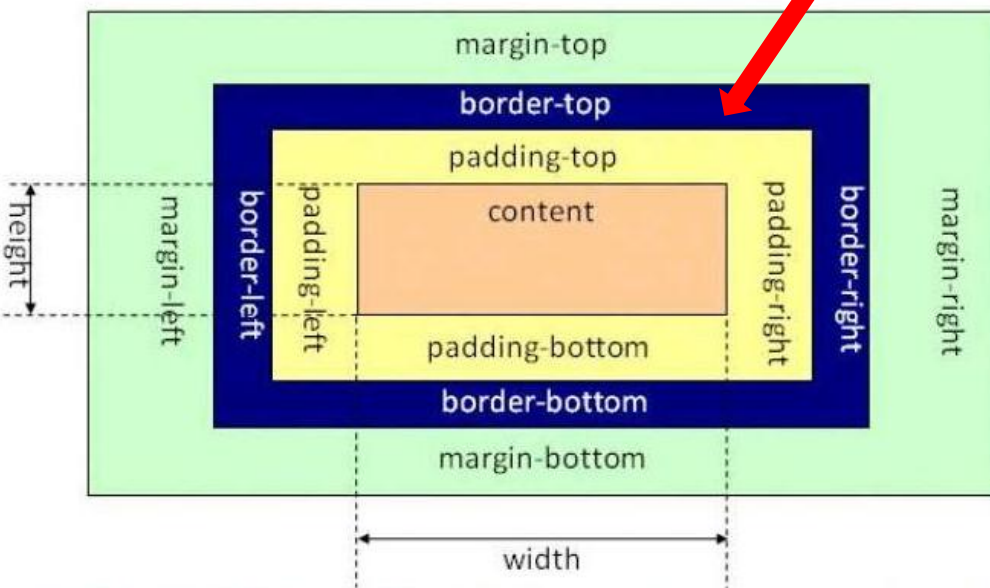
```
h1 {padding: 10px 0.25em 2ex 20%;}
```

顺序：top-right-bottom-left
顺时针方向



盒模型

- 元素框的最内部分是实际的内容，直接包围内容的是内边距，内边距的边缘是边框，边框以外是外边距

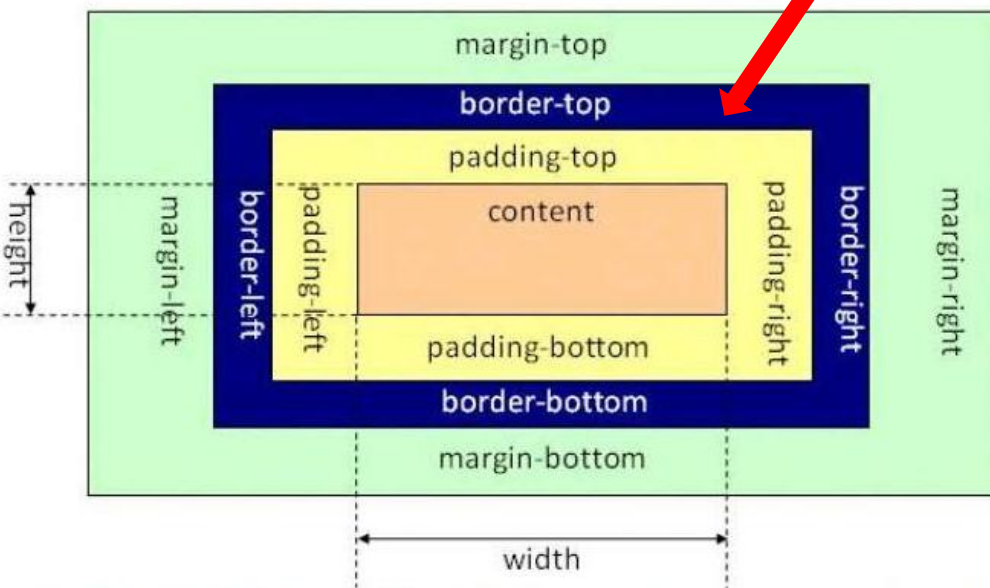


border表示**边框**，默认值为0，可以设置宽度、风格和颜色

- 通过border 简写属性可以一次设置所有四个边
- 通过border-top, border-right, border-bottom, border-left 分别设置某边的厚度、风格和颜色
- 通过border-width, border-style, border-color单独设置厚度、风格和颜色并应用到全部四边
- 可单独设置某边的三个不同属性，如 border-top-width, border-top-style, border-top-color

盒模型

- 元素框的最内部分是实际的内容，直接包围内容的是内边距，内边距的边缘是边框，边框以外是外边距

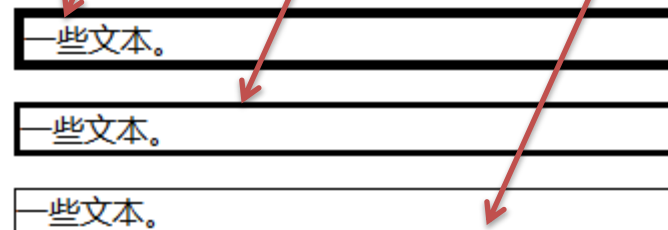


border表示**边框**，默认值为0，可以设置宽度、风格和颜色

```
border-style:solid;  
border-width:5px;
```

```
border-style:solid;  
border-width:medium;
```

```
border-style:solid;  
border-width:1px;
```



注意: "border-width" 属性 如果单独使用则不起作用。要先使用 "border-style" 属性来设置边框。

盒模型

- 元素框的最内部分是实际的内容，直接包围内容的是内边距，内边距的边缘是边框，边框以外是外边距

border 表示**边框**，默认值为0，可以设置宽度、风格和颜色

border-style 值:

none: 默认无边框

border-style:none;

dotted: 定义一个点线边框

border-style:dotted;

dashed: 定义一个虚线边框

solid: 定义实线边框

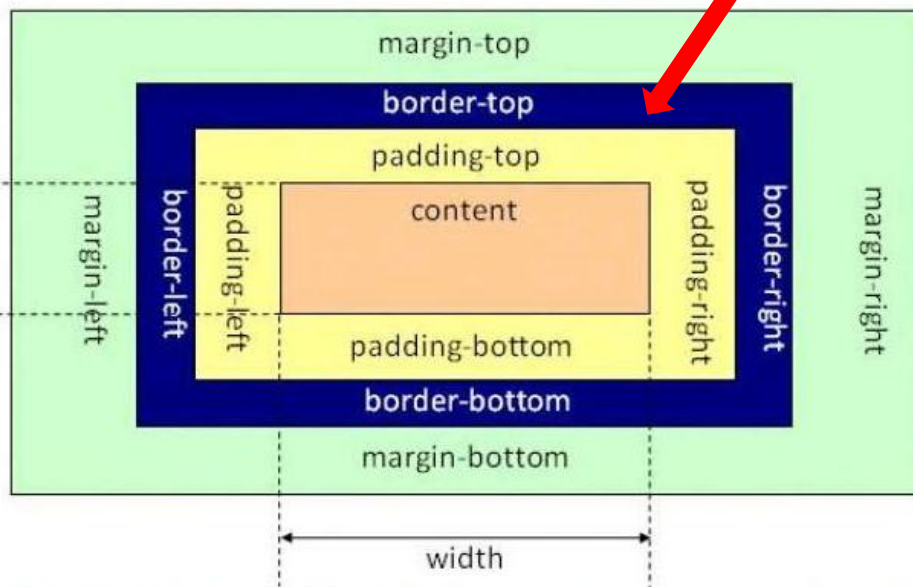
double: 定义两个边框。两个边框的宽度和 border-width 的值相同

groove: 定义3D沟槽边框。效果取决于边框的颜色值

ridge: 定义3D脊边框。效果取决于边框的颜色值

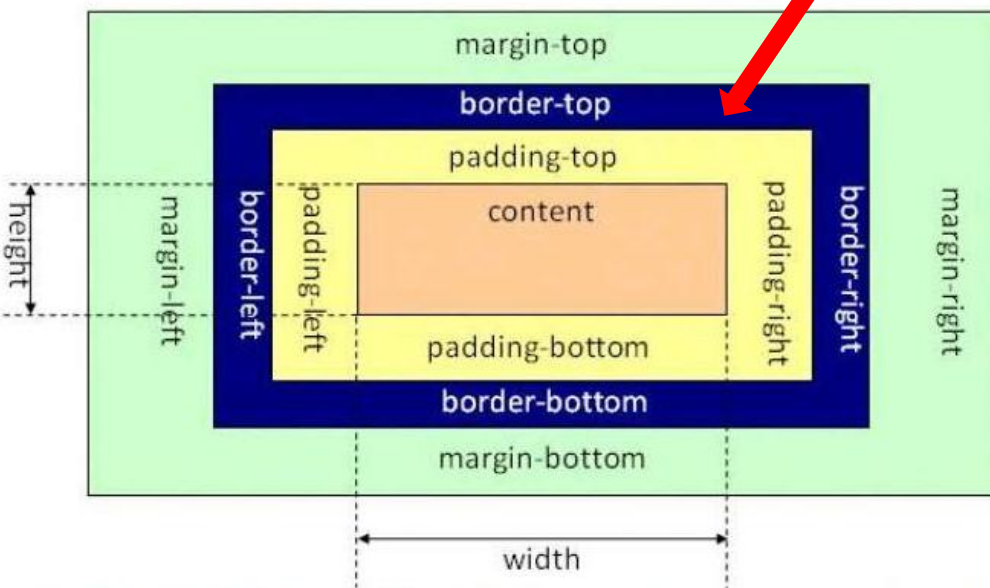
inset: 定义一个3D的嵌入边框。效果取决于边框的颜色值

outset: 定义一个3D突出边框。效果取决于边框的颜色值



盒模型

- 元素框的最内部分是实际的内容，直接包围内容的是内边距，内边距的边缘是边框，边框以外是外边距



border 表示**边框**，默认值为0，可以设置宽度、风格和颜色

顺序：**top-right-bottom-left**

```
p {  
  border-style: solid;  
  border-color: blue rgb(25%,35%,45%) #909090 red;  
}
```

```
p {  
  border-style: solid;  
  border-color: blue red;  
}
```

少于4个属性
值复制

top right bottom left

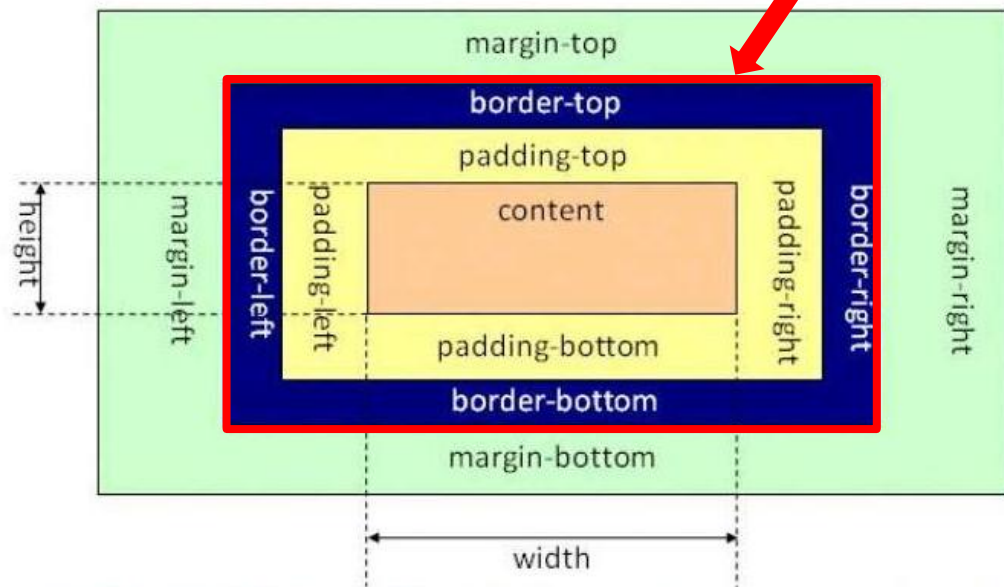
上下边框是蓝色
左右边框是红色

默认的边框颜色是元素本身的前景色。如果没有为边框声明颜色，它将与元素的文本颜色相同。如果元素没有任何文本，假设它是一个表格，其中只包含图像，那么该表的边框颜色就是其父元素的文本颜色（因为 color 可以继承）

盒模型

- 元素框的最内部分是实际的内容，直接包围内容的是内边距，内边距的边缘是边框，边框以外是外边距

outline



border 表示**边框**，默认值为0，可以设置宽度、风格和颜色

outline（**轮廓**）是绘制于**元素周围**的一条线，位于边框边缘的外围，可起到突出元素的作用，可以通过 `outline-color`, `outline-style`, `outline-width` 设置样式。（设置div 宽高都是100px，加了border以后变成102，outline还是100）

border 更加灵活，可以指定边框的宽度、类型、颜色和圆角效果；而 outline 则更适合用于表示元素的状态，例如：`hover`、`focus` 或 `active` 等。

盒模型

- 背景裁剪 (background clip)

— 背景色、背景图片默认应用于由内容和内边距、边框组成的区域，可以通过设置background-clip属性值来调整

```
<div class="default"></div>
<div class="padding-box"></div>
<div class="content-box"></div>
```

```
div {
  width : 60px;
  height : 60px;
  border : 20px solid rgba(0, 0, 0, 0.5); 半透明黑色
  padding : 20px;
  margin : 20px 0;

  background-size : 140px;
  background-position : center;
  background-image : url('https://mdn.mozillademos.org/files/11947/ff-logo.png');
  background-color : gold;
}

.default { background-clip : border-box; }
.padding-box { background-clip : padding-box; }
.content-box { background-clip : content-box; }
```

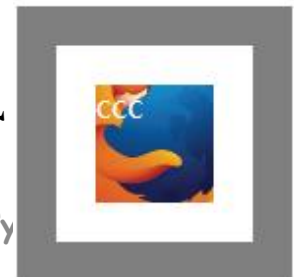
默认到边框



填充到内边距



填充到内容区域

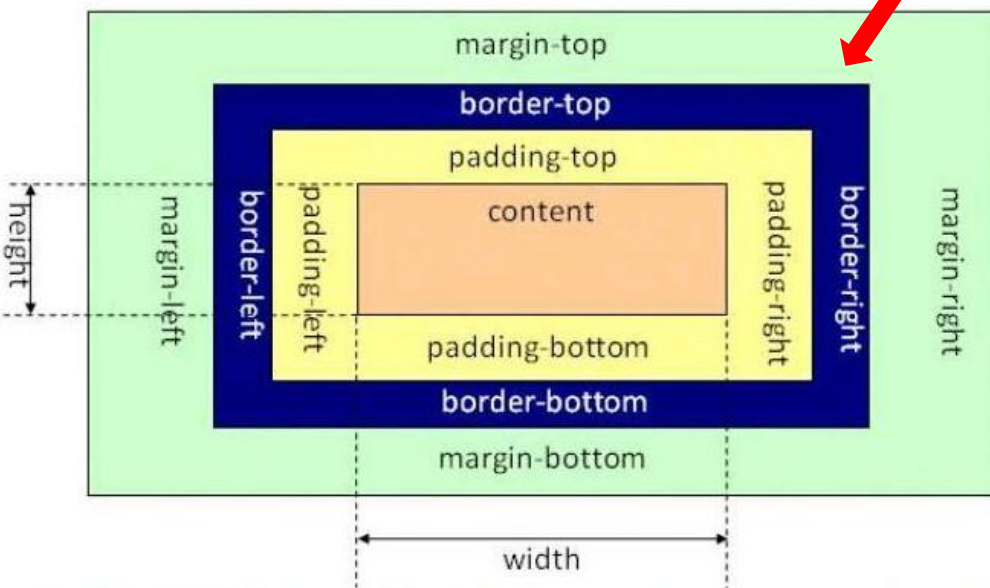


盒模型

- 元素框的最内部分是实际的内容，直接包围内容的是内边距，内边距的边缘是边框，边框以外是外边距

margin表示**外边距**，默认值为0，在CSS布局中**推开**其它盒子

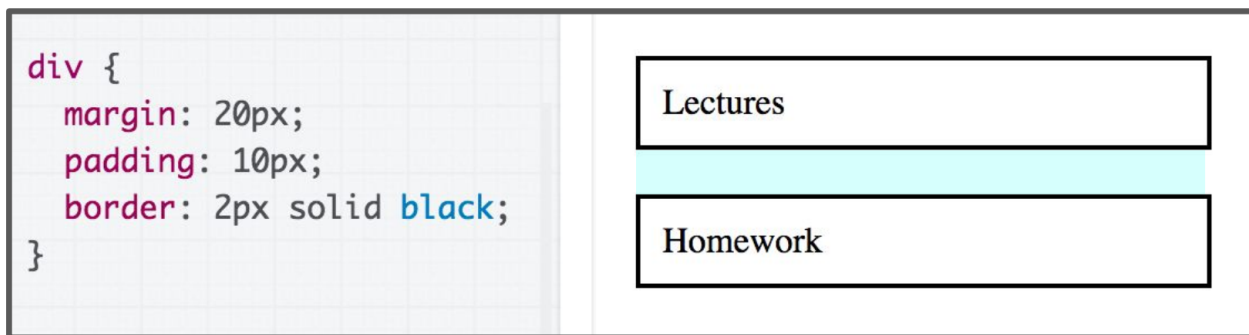
- 可以通过margin或 margin-top、margin-right、margin-bottom 和 margin-left一次性或分布设置宽度、风格和颜色。
- 外边距默认是透明的，不会遮挡其后的任何元素



盒模型

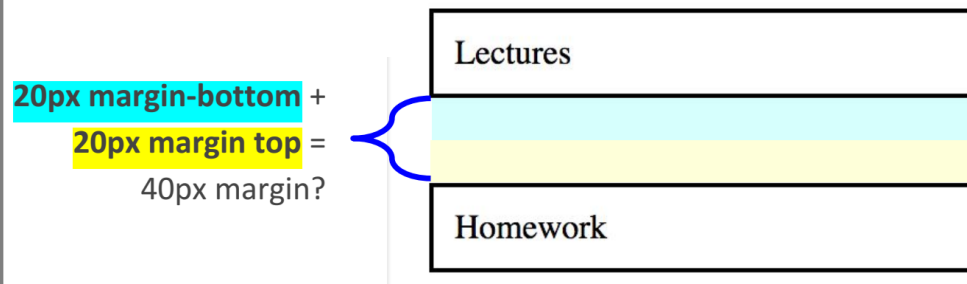
- 外边距重叠 (collapsing margins)

– 相邻+块级元素的上外边距和下外边距有时会合并为一个外边距，其大小取其中的最大者，这种行为称为外边距折叠



问：两个<div>元素之间的距离有多远？是 $20+20=40\text{px}$ 吗？

答：错，存在外边距重叠现象，实际是 $\max(20,20)=20\text{px}$

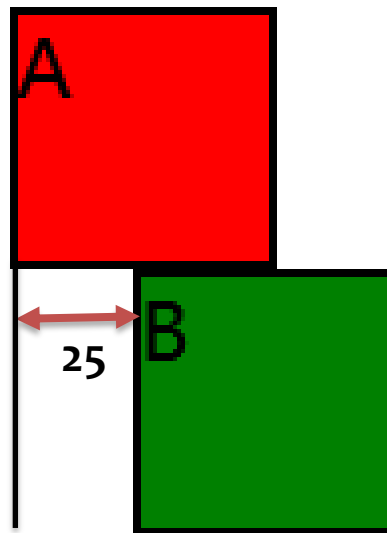


盒模型

- 外边距重叠 (collapsing margins)
 - 初始状态，外边距为0，两个盒子紧贴在一起

```
<div class="BoxA">A</div>  
<div class="BoxB">B</div>
```

```
.BoxA {  
  height: 50px;  
  margin-bottom: 0px;  
  border: 2px solid black;  
  background-color: red;  
  width: 50px;  
}  
  
.BoxB {  
  height: 50px;  
  margin-top: 0px;  
  margin-left: 25px;  
  border: 2px solid black;  
  background-color: green;  
  width: 50px;  
}
```



盒模型

- 外边距重叠 (collapsing margins)

- 在参与折叠的margin都是正数的情况下，取其中较大的值为最终margin值

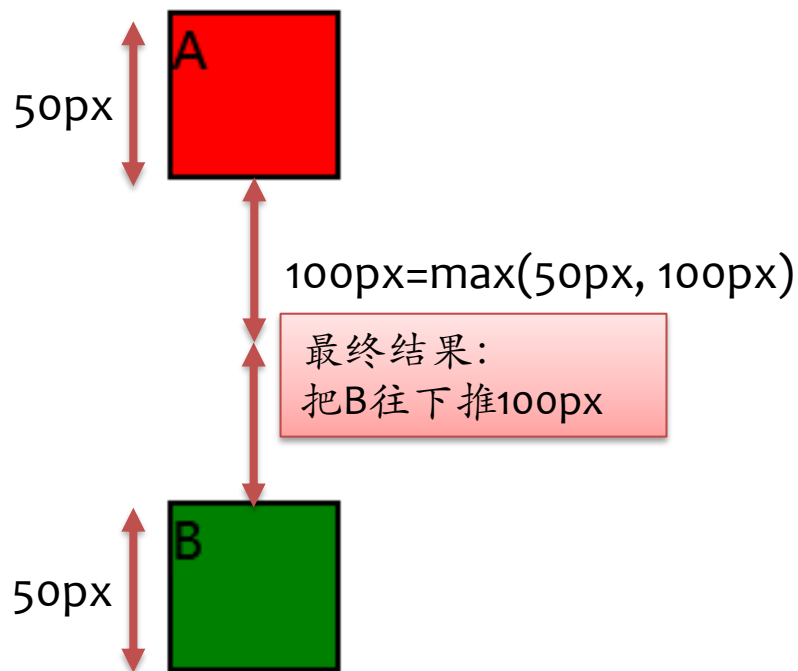
```
<div class="BoxA">A</div>  
<div class="BoxB">B</div>
```

```
.BoxA {  
  height: 50px;  
  margin-bottom: 50px;  
  border: 2px solid black;  
  background-color: red;  
  width: 50px;  
}
```

把B往下推50px

```
.BoxB {  
  height: 50px;  
  margin-top: 100px;  
  border: 2px solid black;  
  background-color: green;  
  width: 50px;  
}
```

把B往下推100px



盒模型

- 外边距重叠 (collapsing margins)

– 在参与折叠的margin都是负值的时候,取其中绝对值较大的,然后从零开始,负向位移

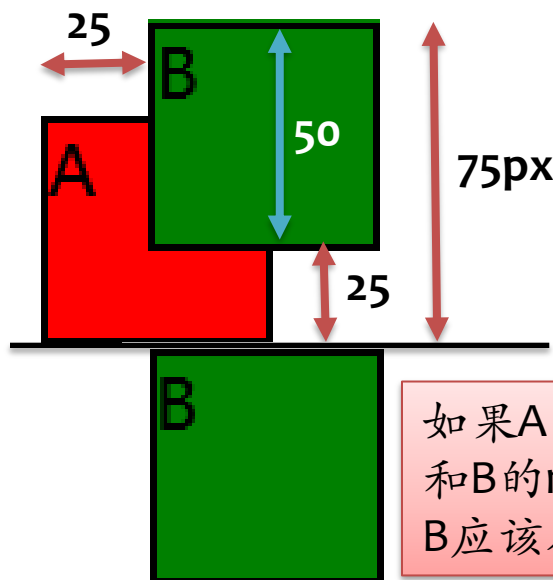
```
<div class="BoxA">A</div>
<div class="BoxB">B</div>
```

```
.BoxA {
  height: 50px;
  margin-bottom: -75px;
  border: 2px solid black;
  background-color: red;
  width: 50px;
}
```

```
.BoxB {
  height: 50px;
  margin-top: -50px;
  margin-left: 25px;
  border: 2px solid black;
  background-color: green;
  width: 50px;
}
```

把B往上拉75px

把B往上拉50px



最终结果:
把B往上拉75px

如果A的margin-bottom
和B的margin-top都是0,
B应该在这个位置

盒模型

- 外边距重叠 (collapsing margins)

- 在参与折叠的margin有正值有负值的时候,要从所有负值中选出绝对值最大的,所有正值中选择绝对值最大的,二者相加, $100\text{px} + (-50\text{px}) = 50\text{px}$

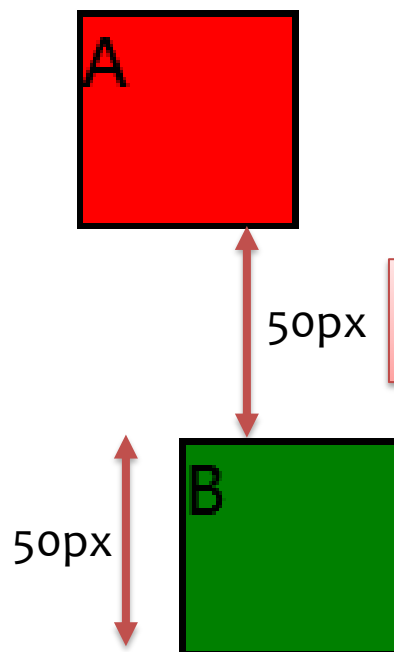
```
<div class="BoxA">A</div>
<div class="BoxB">B</div>
```

```
.BoxA {
  height: 50px;
  margin-bottom: -50px;
  border: 2px solid black;
  background-color: red;
  width: 50px;
}
```

```
.BoxB {
  height: 50px;
  margin-top: 100px;
  margin-left: 25px;
  border: 2px solid black;
  background-color: green;
  width: 50px;
}
```

把B往上拉50px

把B往下推100px



最终结果:
把B往下推50px

盒模型

- 自动外边距 (auto margins)
 - 对于块级元素，如果将其margin-left和margin-right设定为auto, 则可将该块级元素居中



This is a box of text.

盒模型

• 内联元素的盒子模型

margin:50px

padding:15px

I am a paragraph and this is a span inside that paragraph. A span is an inline element and so does not respect width and height.

- ① 设置宽度和高度未生效，外边距、内边距和边框生效
- ② 不会改变其他内容与内联盒子的关系，因此内边距和边框会与段落中的其他单词重叠

Interactive editor

```
span {  
  margin: 50px;  
  padding: 15px;  
  width: 800px;  
  height: 500px;  
  background-color: lightblue;  
  border: 2px solid blue;  
}
```

为span元素（内联元素）设置了宽度、高度、边距、边框和内边距

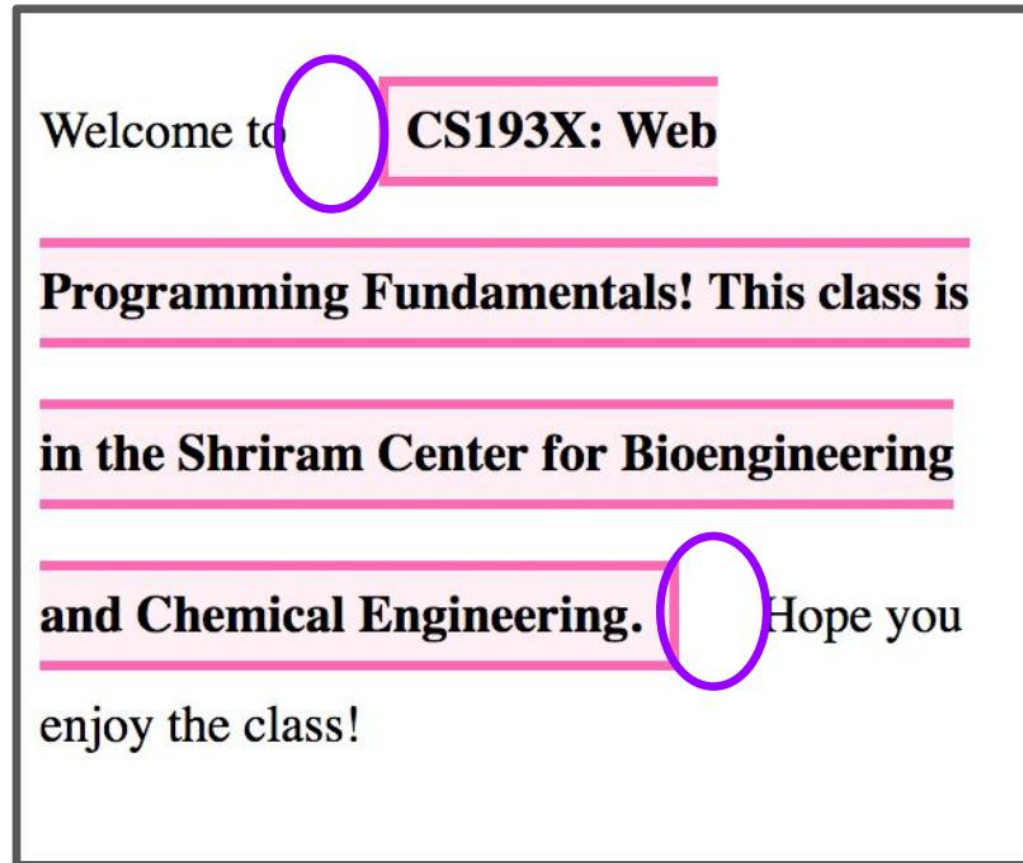
```
<p>  
  I am a paragraph and this is a <span>span</span> inside that paragraph. A  
  span is an inline element and so does not respect width and height.  
</p>
```

盒模型

- 内联元素的盒子模型

```
CSS
strong {
  border: 3px solid hotpink;
  padding: 5px;
  margin: 25px;
  line-height: 50px;
  background-color: lavenderblush;
}
```

- **margin** is to the left and right of the inline element
 - margin-top and margin-bottom are **ignored**
- use **line-height** to manage space between lines



([Codepen](#))

盒模型

• 内联块元素的盒子模型

margin:10px

Padding:10px

W*H: 80*50

I am a paragraph and this is a

span

inside that

paragraph. A span is an inline element and so does not respect width and height.

一个元素使用 **display: inline-block**，实现块级的部分效果：

①设置**width**和**height**属性会生效

②padding, margin, 以及border 会推开其他元素但是：

③它**不会跳转到新行**，如果显式添加**width**和**height**属性，它只会变得比其实际内容更大

Interactive editor

```
span {  
  margin: 10px;  
  padding: 10px;  
  width: 80px;  
  height: 50px;  
  background-color: lightblue;  
  border: 2px solid blue;  
  display: inline-block;  
}
```

为span元素（内联块元素）设置了宽度、高度、边距、边框和内边距

```
<p>  
  I am a paragraph and this is a <span>span</span> inside that paragraph. A  
  span is an inline element and so does not respect width and height.  
</p>
```


盒模型

- body元素也有缺省的margin

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <link rel="stylesheet" type="text/css" href="css/style.css">
    <title>二次元俱乐部</title>
  </head>
  <body>
    <div class="testbodymargin">本段内容用以测试body的margin</div>
```

```
.testbodymargin{
  background-color: aqua;
  height: 50px;
}
```

本段内容用以测试body的margin

聆听电子天籁之音

- [主页](#)

要搜索的内容

相关链接

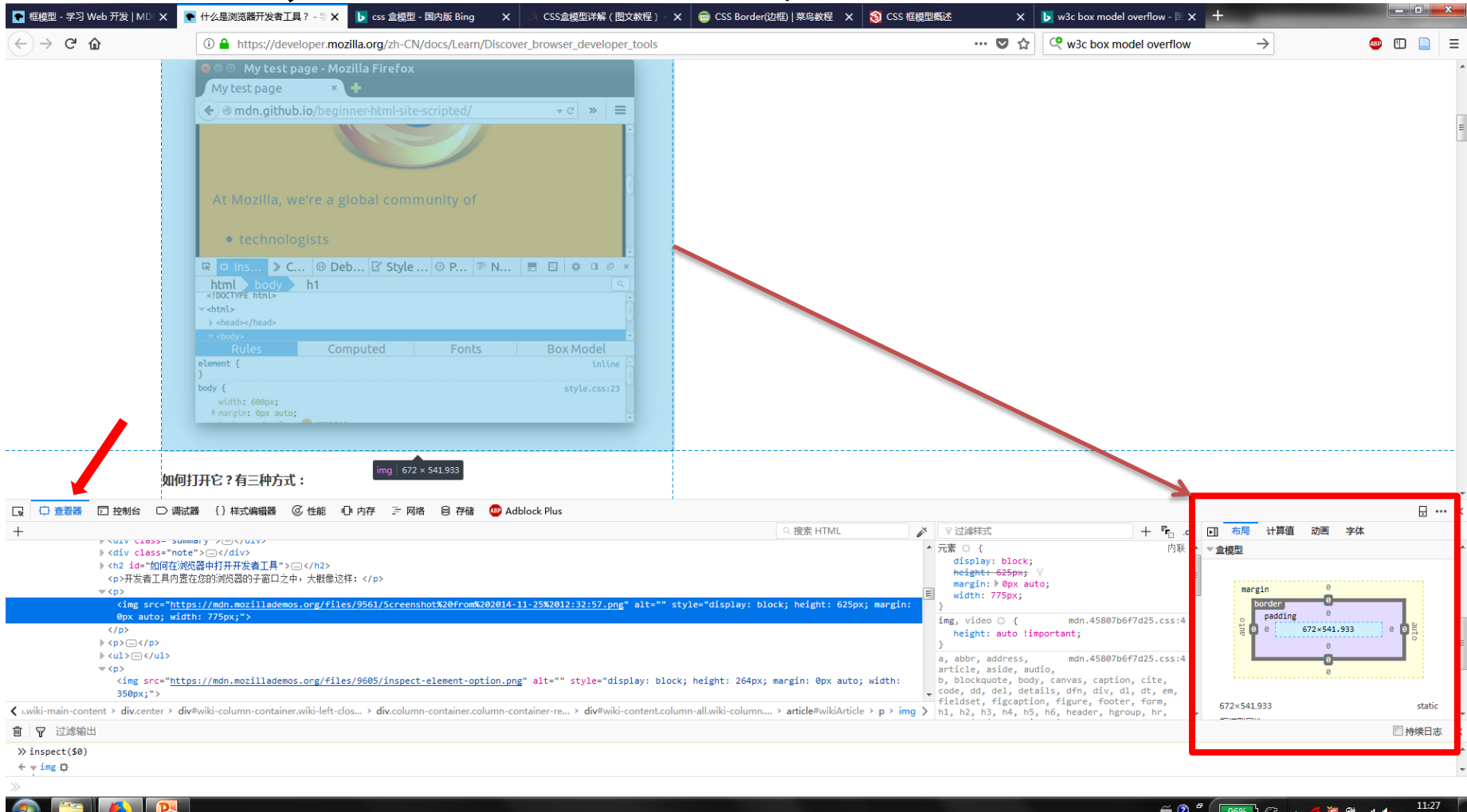
- [这是一](#)

本段内容用以测试b

为什么div的周边还有一圈空白？

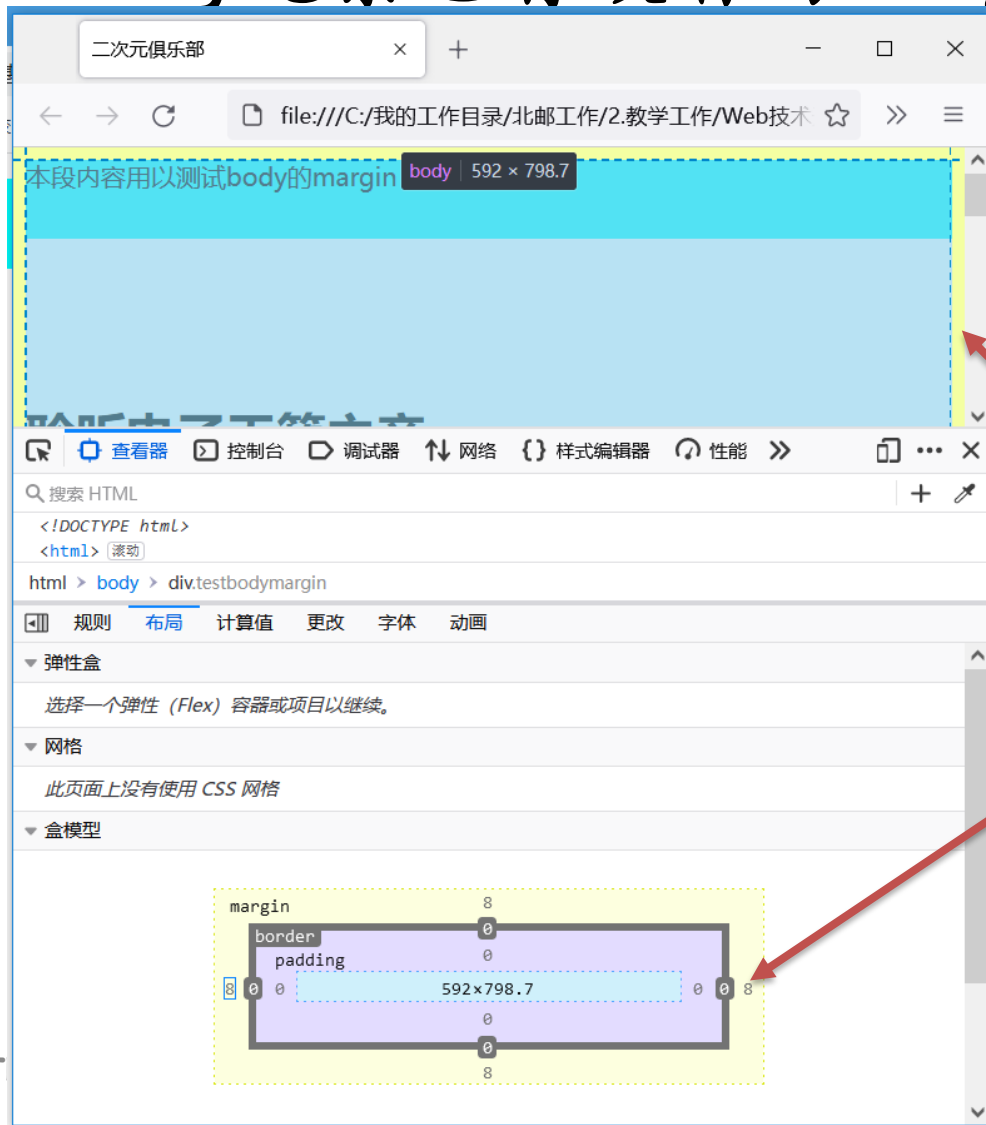
盒模型

- 通过打开浏览器开发工具并单击DOM检查器中的元素，查看页面上各个元素的盒模型



盒模型

- body元素也有缺省的margin



利用浏览器的Web开发者工具
查看body的盒子模型
发现其存在margin:8px的默认值

盒模型

- body元素也有缺省的margin

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <link rel="stylesheet" type="text/css" href="css/style.css">
    <title>二次元俱乐部</title>
  </head>
  <body>
    <div class="testbodymargin">本段内容用以测试body的margin</div>
```

```
.testbodymargin{
  background-color: aqua;
  height: 50px;
}

body{
  margin: 0px;
}
```

本段内容用以测试body的margin

聆听电子天籁之音

- [主页](#)

要搜索的内容

搜索

相关链接

- [这是一个超链接](#)

div周边的空白不见了，
是由body的margin引起的

盒模型

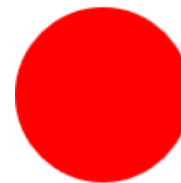
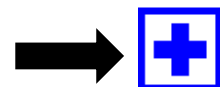
• 有关边框的一些小技巧【课后动手实验】

- 同一元素的边框相交处是斜线，可用来实现三角形
- border-radius可以为边框设置圆角(IE8-不支持)，四值顺序是左上、右上、右下、左下

```
<div class="box1"></div>
<div class="box2"></div>
<div class="box3"></div>
<div class="box4"></div>
<div class="box5"></div>
<div class="box6"></div>
```



```
<style>
.box{
  position: relative;
  color: blue;
  border: 4px solid;
  width: 40px;
  height: 40px;
  transition: color 0.2s;
}
.box::before{
  content: "";
  border-top: 10px solid;
  display: block;
  position: absolute;
  width: 30px;
  top: 15px;
  left: 5px;
}
.box::after{
  content: "";
  position: absolute;
  top: 5px;
  left: 15px;
  border-left: 10px solid;
  height: 30px;
}
.box:hover{
  color: lightblue;
}
</style>
```



盒模型

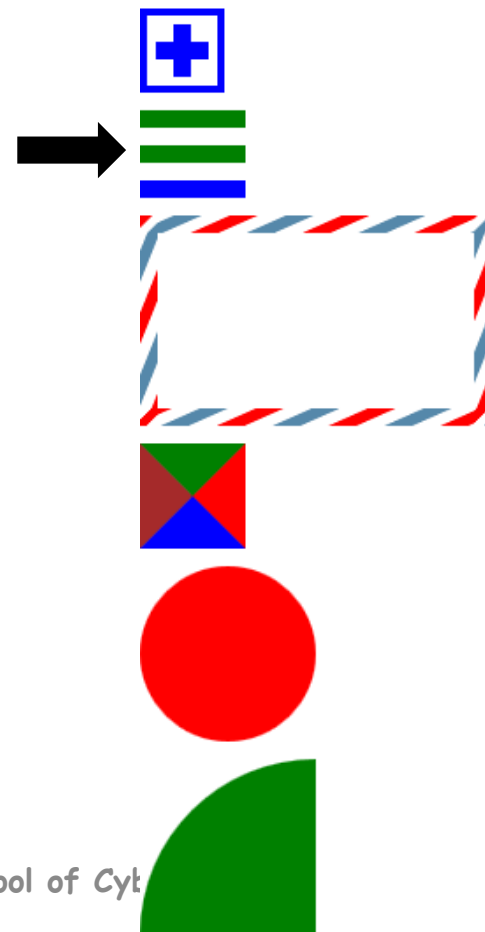
• 有关边框的一些小技巧【课后动手实验】

- 同一元素的边框相交处是斜线，可用来实现三角形
- border-radius可以为边框设置圆角(IE8-不支持)，四值顺序是左上、右上、右下、左下

```
<div class="box1"></div>
<div class="box2"></div>
<div class="box3"></div>
<div class="box4"></div>
<div class="box5"></div>
<div class="box6"></div>
```



```
.box2{
  width: 60px;
  height: 10px;
  border-top: 30px double green;
  border-bottom: 10px solid blue;
  margin-top: 10px;
}
```



盒模型

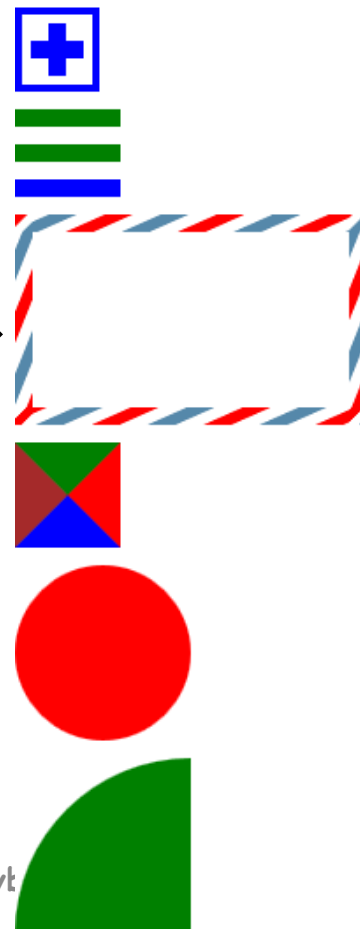
• 有关边框的一些小技巧【课后动手实验】

- 同一元素的边框相交处是斜线，可用来实现三角形
- border-radius可以为边框设置圆角(IE8-不支持)，四值顺序是左上、右上、右下、左下

```
<div class="box1"></div>
<div class="box2"></div>
<div class="box3"></div>
<div class="box4"></div>
<div class="box5"></div>
<div class="box6"></div>
```



```
.box3 {
  width: 160px;
  height: 80px;
  padding: 10px;
  border: 10px solid;
  border-image: repeating-linear-gradient(-225deg, red 0, red 10px,
    transparent 10px, transparent 20px, #58a 20px, #58a 30px,
    transparent 30px, transparent 40px) 20;
  margin-top: 10px;
}
```



盒模型

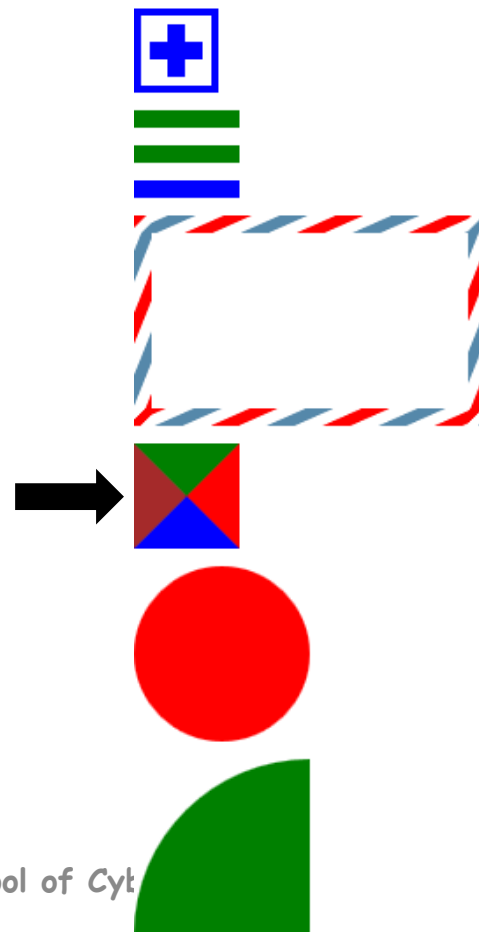
• 有关边框的一些小技巧【课后动手实验】

- 同一元素的边框相交处是斜线，可用来实现三角形
- border-radius可以为边框设置圆角(IE8-不支持)，四值顺序是左上、右上、右下、左下

```
<div class="box1"></div>
<div class="box2"></div>
<div class="box3"></div>
<div class="box4"></div>
<div class="box5"></div>
<div class="box6"></div>
```



```
.box4{
  width: 0px;
  height: 0px;
  border: 30px solid white;
  border-top-color: green;
  border-bottom-color: blue;
  border-left-color: brown;
  border-right-color:red;
  margin-top:10px;
  /*border-right:none;*/try this*/
}
```



盒模型

• 有关边框的一些小技巧【课后动手实验】

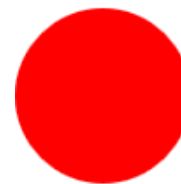
- 同一元素的边框相交处是斜线，可用来实现三角形
- border-radius可以为边框设置圆角(IE8-不支持)，四值顺序是左上、右上、右下、左下

```
<div class="box1"></div>
<div class="box2"></div>
<div class="box3"></div>
<div class="box4"></div>
<div class="box5"></div>
<div class="box6"></div>
```



```
.box5{
  background-color:red;
  width: 100px;
  height: 100px;
  border-radius: 50%;
  margin-top:10px;
}

.box6{
  background-color:green;
  width: 100px;
  height: 100px;
  border-radius: 100px 0 0 0;
  margin-top:10px;
}
```



网安小知识--朝鲜方向

拉撒路组织--Lazarus

组织背景

- 隶属于朝鲜侦查总局下属121局
- 1994年派15名朝鲜人到国外的一所军事学院学习黑客技术
- 军方从1996年开始认真训练计算机“战士”，两年后开设了121局



组织成员

- 2018年9月，美国联邦调查局起诉拉撒路组织成员朴镇赫
- 2021年2月，美国联邦调查局起诉了另外两名该组织成员：金日、全昌赫



技术能力

- 极富创意的网络入侵能力，曾针对全球网络安全分析师开展钓鱼攻击活动
- 极强的反侦察能力。组织成员大多在中国沈阳、大连及俄罗斯境内开展网络作战活动，并且善于使用中国化名



典型事件

- 2017年5月，该组织利用美军方泄露的永恒之蓝漏洞，面向全球发起WannaCry勒索攻击
- 2014年索尼影业入侵事件
- 2016年2月，孟加拉央行大劫案

